

Git Pain Points & Solutions for Researchers

A quick-reference guide for common issues that come up when using Git in research workflows.

1. Authenticating with GitHub from the Command Line

GitHub no longer accepts passwords for command-line Git operations. You have three options:

Option A: GitHub CLI (recommended for beginners)

```
# Install
brew install gh          # macOS
winget install GitHub.cli # Windows

# Authenticate (interactive – walks you through it)
gh auth login
```

This handles everything: HTTPS tokens, SSH keys, and even sets Git's credential helper for you. After running `gh auth login`, normal `git push` / `git pull` just work.

Option B: Personal Access Token (PAT)

1. Go to **GitHub** → **Settings** → **Developer Settings** → **Personal Access Tokens** → **Tokens (classic)**
2. Generate a token with `repo` scope
3. When Git asks for your password, paste the token instead
4. To avoid re-entering it every time:

```
# Cache credentials for 8 hours (28800 seconds)
git config --global credential.helper 'cache --timeout=28800'

# Or store permanently (saved in plain text – less secure)
git config --global credential.helper store
```

macOS uses Keychain automatically. **Windows** can use:

```
git config --global credential.helper manager
```

Option C: SSH Keys

```
# Generate a key pair
ssh-keygen -t ed25519 -C "you@example.com"

# Copy public key to clipboard
cat ~/.ssh/id_ed25519.pub # then paste into GitHub → Settings → SSH Keys

# Test
ssh -T git@github.com
```

Then clone/add remotes using the SSH URL (`git@github.com:user/repo.git`) instead of HTTPS.

Which should I recommend to participants?

Audience	Recommendation
Brand new to Git	GitHub Desktop (handles auth automatically)
Comfortable with terminal	GitHub CLI (<code>gh auth login</code>)
Power users / HPC	SSH keys

2. Managing Large / Data Files

Git stores every version of every file. A 500 MB CSV that changes 10 times = ~5 GB repo. This breaks things fast.

Rule of thumb

File size	Strategy
< 1 MB	Track normally with Git
1–100 MB	Consider <code>.gitignore</code> + separate storage, or Git LFS
> 100 MB	Do not put in Git. Use Git LFS, git-annex, or external storage

Option A: Just don't track it

The simplest approach. Add data paths to `.gitignore` and document where to get the data:

```
data/raw/
data/external/
*.h5
```

```
*.nwb  
*.tif
```

Include a `README.md` or `data/README.md` explaining:

- Where the data lives (e.g., server path, Dropbox, Zenodo DOI)
- How to download or regenerate it
- A small sample dataset *can* be committed for testing

Option B: Git LFS (Large File Storage)

Replaces large files with lightweight pointers in Git, stores the actual content on a separate server.

```
# Install  
brew install git-lfs      # macOS  
git lfs install           # one-time setup  
  
# Track file types  
git lfs track "*.csv"  
git lfs track "*.h5"  
  
# This creates/updates .gitattributes – commit it!  
git add .gitattributes  
git commit -m "Track large files with LFS"  
  
# Then use git normally  
git add data/experiment.csv  
git commit -m "Add raw experiment data"  
git push origin main
```

Limitations:

- GitHub free: 1 GB storage, 1 GB/month bandwidth
- Every collaborator needs `git lfs install`
- Not ideal for thousands of large files

Option C: git-annex / DataLad (for serious data management)

Best for neuroscience / large-scale datasets. DataLad wraps git-annex with a more user-friendly interface.

```
# Install  
pip install datalad  
  
# Create a dataset
```

```
datalad create my-project
cd my-project

# Add large files (annexed automatically)
datalad save -m "Add raw recordings" data/
```

Hosting options for annexed data:

- **GIN (G-Node Infrastructure)** — free, designed for scientific data, supports git-annex natively
- **DataLad Hub** — powered by Forgejo-aneksajo
- Any SSH server, S3 bucket, etc.

Comparison

	Git LFS	git-annex / DataLad
Setup complexity	Low	Medium–High
Best for	A few large files	Many large files, complex datasets
Hosting	GitHub, GitLab	GIN, any SSH server, S3
Deduplication	No	Yes
Partial clone (get only some files)	Limited	Yes
Research community adoption	General	Neuroscience, psychology

3. Notebooks (Jupyter, R Markdown)

Notebooks are JSON files (Jupyter) or mixed text (Rmd) that embed code, output, and sometimes base64-encoded images. This makes diffs noisy and merges painful.

The core problem

```
// A single-pixel change in a plot output looks like this in git diff:
- "image/png": "iVBORw0KGgoAAAANSUhEUgAAA... (500 lines of base64)"
+ "image/png": "iVBORw0KGgoAAAANSUhEUgAAB... (500 lines of base64)"
```

Strategies

Strategy 1: Clear outputs before committing (simplest)

Jupyter:

```
# One-time: install nbstripout
pip install nbstripout

# Enable for this repo (auto-strips on commit)
nbstripout --install

# Or manually clear before committing
jupyter nbconvert --ClearOutputPreprocessor.enabled=True --inplace notebook.ipynb
```

R Markdown: Outputs are typically separate files (HTML, PDF), so just `.gitignore` them:

```
*.html
*.pdf
*_files/
```

Strategy 2: Pair notebooks with plain scripts

Keep a `.py` or `.R` script as the "source of truth" and treat the notebook as a rendered view:

```
# Using jupyter (auto-syncs .ipynb ↔ .py)
pip install jupyter
jupyter --set-formats ipynb,py:percent notebook.ipynb
```

Then commit the `.py` file and `.gitignore` the `.ipynb`, or commit both.

Strategy 3: Review tools for notebooks

- **nbdime** — diff and merge tool designed for Jupyter notebooks:

```
pip install nbdime
nbdime config-git --enable      # integrates with git diff/merge

nbdiff notebook_v1.ipynb notebook_v2.ipynb      # terminal diff
nbdiff-web notebook_v1.ipynb notebook_v2.ipynb  # visual diff in browser
```

- **GitHub** renders `.ipynb` files natively (but diffs are still noisy)
- **VS Code** has built-in notebook diffing

Recommended workflow for the session

Approach	When to use
<code>nbstripout</code>	Default — always install this in research repos
<code>jupytertext</code>	When you want clean diffs <i>and</i> notebook convenience
<code>nbdime</code>	When collaborating on notebooks and need to review/merge

4. Binary Files (.docx, .pdf, images)

Git can store binary files, but it can't show meaningful diffs — every change looks like "binary files differ."

What to do

File type	Recommendation
<code>.docx</code> , <code>.pptx</code>	Avoid in Git. Use Google Docs, Overleaf, or convert to Markdown/LaTeX
<code>.pdf</code> (generated)	<code>.gitignore</code> — regenerate from source
<code>.pdf</code> (archival, e.g., signed forms)	Track with Git or LFS, accept you won't get diffs
Images (<code>.png</code> , <code>.jpg</code>)	Track if small and infrequent. Use LFS if they change often
<code>.fig</code> , <code>.ai</code> , <code>.svg</code>	<code>.svg</code> is text-based (Git-friendly). Others are binary — use LFS

Enabling partial diffs for some binary formats

You can teach Git to extract text from binary files for diffing:

```
# .docx diffs (requires pandoc)
# Add to .gitattributes:
*.docx diff=word

# Add to .git/config or ~/.gitconfig:
[diff "word"]
    textconv = pandoc --to=plain
```

```
# PDF diffs (requires pdftotext from poppler)
*.pdf diff=pdf
```

```
[diff "pdf"]
  textconv = pdftotext -layout
```

These won't be perfect, but they let `git log -p` show *something* meaningful.

Practical advice for the session

"If it's generated, don't track it. If it's source, track it. If it's binary source, consider whether a text-based alternative exists."

```
# Generated outputs – don't track
*.pdf
*.docx
figures/*.png

# But DO track source
# manuscript.tex ← yes
# analysis.R ← yes
# figures/plot.R ← yes (generates the .png)
```

5. Merge Conflicts

Merge conflicts happen when two branches (or two people) edit the same lines of the same file. Git doesn't know which version to keep, so it asks you.

What a conflict looks like

```
The sample size was
<<<<<< HEAD
n = 45 after filtering outliers (>3 SD).
=====
n = 50 including all participants.
>>>>>> new-analysis
```

- `<<<<<< HEAD` — what's in your current branch
- `=====` — separator
- `>>>>>> new-analysis` — what's in the incoming branch

How to resolve

Step 1: See which files are conflicted

```
git status
# Shows: "both modified: analysis.R"
```

Step 2: Open the file and pick what to keep

Edit the file to remove the conflict markers and keep the correct content:

```
The sample size was n = 45 after filtering outliers (>3 SD).
```

Step 3: Mark as resolved and commit

```
git add analysis.R
git commit -m "Resolve conflict: keep filtered sample size"
```

Visual merge tools (easier than editing raw markers)

```
# Use VS Code as your merge tool
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait --merge $REMOTE $LOCAL $BASE $MERGED'

# When a conflict happens:
git mergetool
```

GitHub Desktop and **VS Code** both highlight conflicts visually and let you click to accept "mine," "theirs," or edit manually.

How to minimize conflicts

Practice

Pull before you start working

Why it helps

You're working on the latest version

Practice	Why it helps
Commit small and often	Smaller changes = smaller conflicts
Use branches for experiments	Keeps <code>main</code> stable
Don't edit the same file simultaneously	Communicate with collaborators
Avoid committing generated files	These change constantly and cause spurious conflicts

Live demo suggestion

Create a conflict on purpose to demystify it:

```
# Setup
git init conflict-demo && cd conflict-demo
echo "Result: 42" > results.txt
git add . && git commit -m "Initial result"

# Branch A
git checkout -b branch-a
echo "Result: 45 (filtered)" > results.txt
git commit -am "Update result with filtering"

# Branch B
git checkout main
git checkout -b branch-b
echo "Result: 50 (all subjects)" > results.txt
git commit -am "Update result with all subjects"

# Merge → conflict!
git checkout main
git merge branch-a      # succeeds
git merge branch-b      # CONFLICT – now resolve it together
```

6. Branch, Merge, Push, Pull, Tag

This is the day-to-day workflow for changing a shared research codebase without making every edit directly on `main`.

Core model

Concept	Research analogy
Branch	A separate workspace for one analysis change, bug fix, or documentation update
Commit	A labeled snapshot of one coherent step

Concept	Research analogy
Merge	Bring the completed branch back into the shared project history
Push	Publish local commits to GitHub or another remote host
Pull	Bring remote commits down to another computer, cluster, or collaborator machine
Tag	Name one exact commit as a milestone: release, manuscript submission, analysis freeze

Clean feature workflow

```
# Start from the shared branch
git switch main
git pull --ff-only

# Create a branch for one focused change
git switch -c feature/config-checklist

# Edit files, then inspect the change
git status
git diff

# Stage and commit
git add docs/config-checklist.md
git commit -m "Add config checklist documentation"

# Merge back to main
git switch main
git merge feature/config-checklist

# Publish to GitHub
git push origin main
```

Pulling on another machine

This is common when moving between a laptop, lab workstation, and cluster login node.

```
cd 2P-proc
git status
git pull --ff-only
```

Use `--ff-only` when teaching beginners because it refuses to create an unexpected merge commit during pull. If it fails, Git is telling you that local and remote histories diverged and need attention.

Tagging milestones

Use tags when you need to recover the exact code state used for an important result.

```
# Annotated tag, recommended for meaningful milestones
git tag -a analysis-freeze-2026-05 -m "Analysis freeze for May 2026 results"

# Share the tag
git push origin analysis-freeze-2026-05

# Later, list or inspect tags
git tag
git show analysis-freeze-2026-05
```

Good tag names:

```
v0.2.0
manuscript-submission-2026-05
analysis-freeze-2026-05
boost-demo-2026-05
```

Practical rules

Rule	Why it helps
Pull before starting work	Avoids building on stale history
One branch per task	Makes review and rollback easier
Commit small coherent changes	Makes the history readable
Read <code>git diff</code> before <code>git commit</code>	Prevents accidental files from entering history
Push when work should be shared or backed up	Makes the work available on other machines
Tag exact states used for results	Supports reproducibility months later

Quick Reference: What Goes Where

— .git/	← Git internals (never touch)
— .gitignore	← Files Git should ignore
— .gitattributes	← LFS tracking, diff drivers
— README.md	← Always have one

```
|— code/
|   |— analysis.R      ← ✓ Track
|   └─ utils.py       ← ✓ Track
|— notebooks/
|   └─ exploration.ipynb ← ⚠ Track with nbstripout
|— data/
|   |— README.md      ← ✓ Track (describes data sources)
|   |— raw/           ← ⚠ .gitignore (large, immutable)
|   └─ processed/     ← ⚠ .gitignore (regenerable)
|— figures/
|   |— plot_figure2.R  ← ✓ Track (source)
|   └─ figure2.png     ← ⚠ .gitignore (generated)
└─ manuscript/
    |— manuscript.tex  ← ✓ Track
    └─ manuscript.pdf  ← ⚠ .gitignore (generated)
```

BOOST — Version Control for Reproducible Research — March 11, 2026